Historical-Data Repair Contracts for Gemma Agents

A Calibrated Resource for Compatibility and Persistence Repair

Guilin Zhang

Abstract

We release eight historical-data repair contracts across six Python projects, with 65 historical files and 252 dependent observations of reading, continued use and persistence. The resource makes acceptance explicit when old formats lose information. Two repairs satisfy an 84-observation numeric contract, although literal reference equality rejects 60 observations for one valid repair. A separate calibration of a known AI SDK defect tests the opposite boundary: rejecting ambiguous authorization while preserving safe legacy approvals. The original Gemma baseline accepts four of 32 repairs. A new, single-deployment comparison completes 64 attempts: exact-text editing accepts 10/32, versus 4/32 for a version-checked line-span interface. All complete-acceptance decisions survive API-free replay. Historical bytes, source patches, negative results and API-free verifiers are released. Code, data and paper: https://github.com/GuilinDev/historical-data-repair-contracts/releases/tag/v0.1.2

1. Introduction

Consider an old file containing a floating-point column. Two producer processes save the same values, one as float32 and the other as float64. Their complete files are identical (Figure 1). An evaluator that requires each original width asks the reader to distinguish inputs it cannot distinguish. Yet accepting any successful load is too weak: a reader can silently narrow a type or discard a later mutation.

Historical-data repair needs an explicit acceptance contract. The test-oracle problem [1] is concrete here: which information survives, which behavior must remain usable, and which alternative outputs are legitimate? Lost numeric width may permit widening; ambiguous authorization may require rejection. We package these decisions with executable checks.

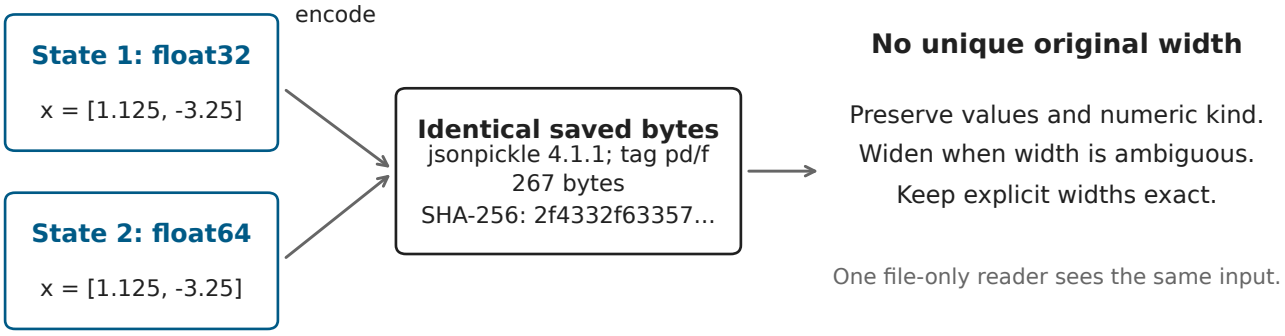


Figure 1. Two historical producer states yield the same complete document. Each state self-roundtrips with its original width, but the saved bytes cannot distinguish them. The pair was found in six targeted producer executions; this is an existence witness, not a collision-rate estimate.

Each repair task supplies an affected reader, producer source, public examples and a source edit scope. A defect lineage groups one upstream defect and its fix with the relevant versions. Files and process repetitions extend coverage; they are not independent bugs. The contribution is eight task bundles, calibrated acceptance boundaries and completed Gemma evaluations. Table 1 summarizes the resource and original baseline.

2. Methods: executable repair contracts

The first four lineages in Table 1 were used during development; four were added prospectively, including two new projects. These tasks come from public projects and may overlap with pretraining data. Genuine releases generated 65 distinct files: 17 public to the agent and 48 withheld from it during repair. All 65 are now released. H/F/R denote input regimes: historical files (131 observations), fresh API construction (53), and current-format roundtrips (68). Each includes declared use and fresh-process reload; they are not successive time steps.

Table 1. Resource, fresh-installation controls and all 32 completed attempts. Model cells count complete semantic and source-policy acceptances; control cells count observations.

	Resource				Controls: pass / N		Gemma: accepted	
Lineage	**Producer; base > fixed**	**Behavior exercised beyond loading**	**Files public / all**	**Checks H / F / R**	**Base**	**Fixed & backport**	**A**	**B**
Development-exposed lineages (4)								
packaging	25.0 / 26.0 26.1 > 26.2	Hash / comparison; collections	2/18	18/6/12	12/36	36/36 each	0/2	0/2
attrs	21.4.0 22.2.0 > 23.1.0	Slot state; cached hash	1/9	9/9/18	27/36	36/36 each	0/2	0/2
jsonpickle CSV	3.3.0 4.1.0 > 4.1.1	CSV text; mutated resave	2/6	6/6/6	12/18	18/18 each	0/2	1/2
SQLAlchemy	1.4.37 1.4.38 > 1.4.39	MutableDict; two commits	2/6	6/6/6	12/18	18/18 each	0/2	0/2
Newly added lineages (4)								
joblib endian	1.2.0 1.4.2 > 1.5.0	Writable mmap; flush	3/7	7/7/7	9/21	21/21 each	0/2	2/2
joblib compressed	1.2.0 1.4.2 > 1.5.0	Compressed load; raw mmap	3/7	7/7/7	9/21	21/21 each	1/2	0/2
jsonpickle dtype	4.1.1 4.1.1 > e7d95a6	Numeric kind / width	2/6	72/6/6	6/84	84/84 each	0/2	0/2
pandas	1.3.5 2.1.0 > 2.1.4	Frame operations; resave	2/6	6/6/6	12/18	18/18 each	0/2	0/2
Total	8 lineages; 6 projects		17/65	131/53/68	99/252	252/252 each	1/16	3/16

Base > fixed denotes affected reader > repaired upstream reader. Dtype H = 6 files x 12 processes; its base score varies by process. e7d95a6 is a pinned upstream commit. A: 26B A4B / PrimeIntellect BF16; B: 31B / CoreWeave FP4.

Contract example: a writable joblib mapping

The agent receives a joblib 1.2.0 file and may edit joblib/numpy_pickle.py (plus an optional test). In this public case, the affected reader changes 23 to 91234 in memory and in the explicit resave, but the work file still contains 23 after flush. Checking both files in new processes detects the lost write-through.

```
work = temporary_copy(historical_file)
a = joblib.load(work, mmap_mode="r+")
assert a.flat[1] == 23
a.flat[1] += 91211
a.flush()
joblib.dump(a, resaved_file)
for path in (work, resaved_file):
    b = fresh_process(joblib.load, path)
    assert b.flat[1] == 91234
```

Simplified from the executed verifier for one public case; fresh_process denotes a new interpreter. The full verifier also checks the remaining declared semantics.

3. Experiments: acceptance boundaries

Numeric ambiguity. In the known jsonpickle defect [2], numeric tags depend on runtime class-address order. Unmodified 4.1.1 producers yield identical files yet self-roundtrip their original widths (Figure 1). Let E_p encode in process p, and x_{32} and x_{64} differ only in numeric width. $E_p(x_{32}) = E_q(x_{64})$: a file-only reader cannot recover both widths exactly.

Bare legacy numeric tags preserve exact values and numeric kind, with nonnarrowing widths allowed. Explicit widths and new encodings remain exact: different outputs need not violate requirements [3].

The conservative backport uses wide types. Document exclusion v2 uses explicit tags in the same document to exclude impossible widths, selecting a unique kind-compatible candidate only for nonempty native-pandas numeric payloads; otherwise it keeps the conservative fallback. A negative control narrows explicit float64 values.

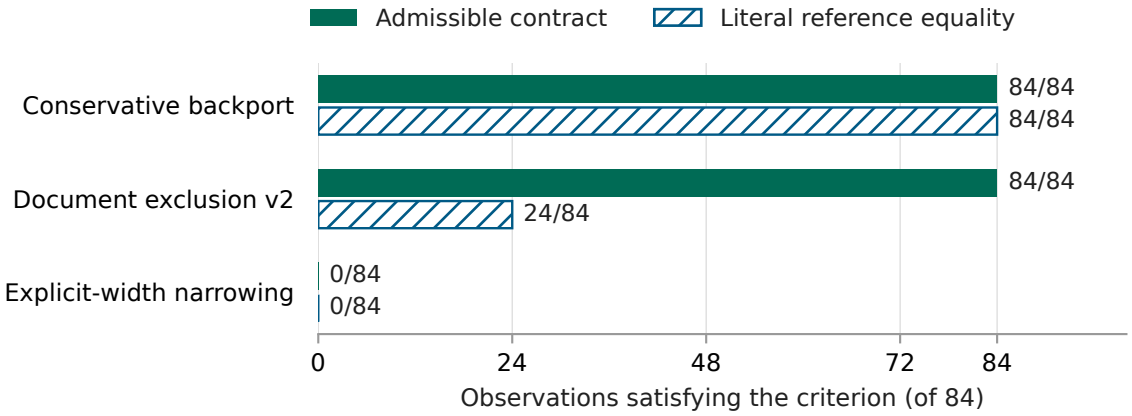


Figure 2. Complete source-repair controls on the numeric-code lineage. All three load and restart 84/84 observations. The alternate valid repair differs from literal reference output on 60 observations (five old files, twelve processes). These are researcher-built controls; literal equality is not the upstream test suite.

Both researcher-written valid policies satisfy 84/84 observations (Figure 2). Literal equality rejects 60 width-only differences: five files, twelve processes. The corrupting control loads and restarts everything but satisfies 0/84. Figure 1 has no disambiguating tags, so both policies retain float64. Failed trial controls remain archived.

Authorization ambiguity. Vercel AI SDK documented newline-joined approval fields yielding identical signed payloads for different tuples [4]. We pin the public v7 fix and its parent. Safe legacy approvals remain valid; legacy identifiers containing a line feed are rejected. New signatures use domain-prefixed JSON arrays. This is known serialization ambiguity, not a discovered HMAC collision.

We froze 54 related scenarios (27 legacy, 27 new-format), including two collision pairs, before testing five implementations (Table 2). Each receives 54 initial and 54 fresh-process verifications, plus separate reissue/reload checks on 37 contract-eligible records, irrespective of each control's actual accepts. The fix passes all scenarios; rejecting all legacy records loses 17 valid cases, while unrestricted fallback accepts five invalid cases. These component probes are not an extra Gemma task or end-to-end authorization. Restart adds no scenario rejections.

Table 2. Signature calibration: scenarios satisfying all declared checks, grouped by expected acceptance. All five implementations complete 54 related scenarios; the bottom three are researcher-written controls.

Implementation	Expected accept	Expected reject	All scenarios
Pre-fix upstream	17/37	12/17	29/54
Upstream v7 fix	37/37	17/17	54/54
Reject all legacy	20/37	17/17	37/54
Unrestricted fallback	37/37	12/17	49/54
Reject all control chars	16/37	17/17	33/54

4. Experiments: Gemma repair attempts

Original baseline. The original 32 Gemma 4 [5] attempts used two conversations per task and deployment: A, 26B A4B via PrimeIntellect BF16; B, 31B via CoreWeave FP4. All calls used OpenRouter, pinned providers and disabled fallback. Provider and precision are confounded, hosted weight revisions are unattested, and public fixes may overlap pretraining. Four repairs pass complete semantics and source review (Table 1); sixteen leave source unchanged. The release retains all traces, including eight same-context rate-limit recoveries and the original attempt matrix.

Controlled interface comparison. Exact-text edits failed 78 times in eleven baseline attempts, but only five of the sixteen unchanged attempts encountered such errors. The competition's edit_file similarly defaults to unique matching [6]; this motivates a test, not a claim about official-harness failure rates. We froze 64 new conversations: eight lineages, four replicates, two interfaces. Task/replicate blocks were randomized, with both arms run together on the same CoreWeave FP4 Gemma 4 31B endpoint.

Both arms expose identical source lines, hashes, contracts and public checks. Exact editing requires one text match; span editing atomically replaces inclusive line ranges against a checked source version. Both validate syntax and allow 24 rounds (4,096 output tokens, temperature 0.2), a 120,000-request-byte guard and a reserved final report (2,048 tokens, temperature zero). Acceptance requires real source change, every frozen observation and source review.

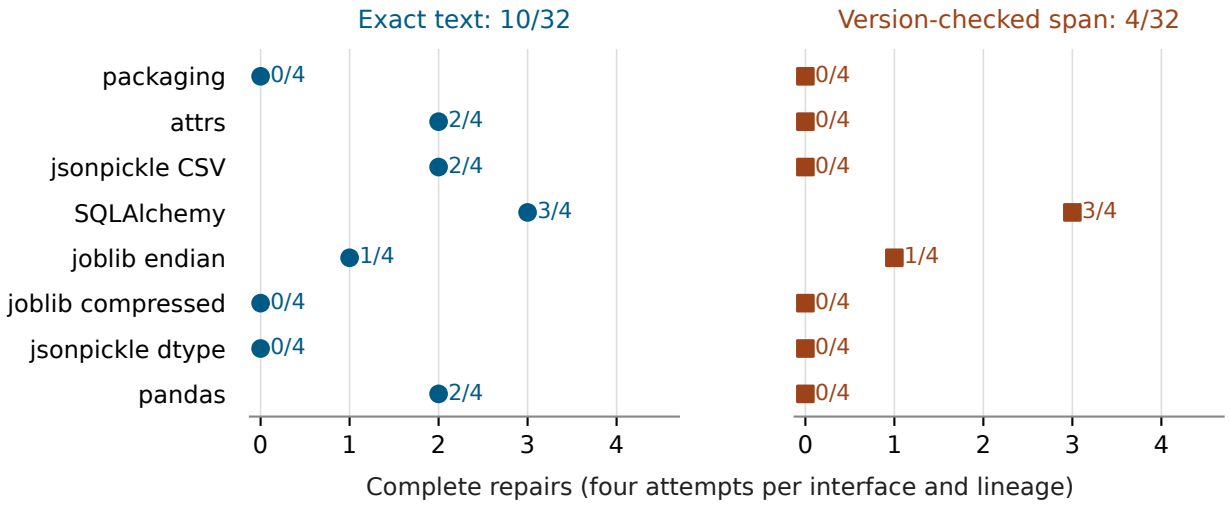


Figure 3. Complete repair acceptance in all 64 new attempts. Each point counts four conversations on one lineage with the same Gemma 4 31B deployment. Circles: exact text, 10/32 total; squares: version-checked spans, 4/32. Repetitions are nested in eight selected lineages; these are observed counts, not a general editor ranking.

All 64 attempts finish and receive decisive evaluations (Figure 3). Exact editing accepts 10/32; the span package accepts 4/32. Unchanged source increases from 6 to 19. Span edits incur 49 syntax/indentation rejections across 15 attempts; independent reconstruction reproduces every rejection. This does not establish how much those errors caused the acceptance gap. The tested interface package did not improve full repairs; eight selected lineages do not establish a general editor ranking.

Two reviewed pauses recover new-file metadata and a pre-network ledger-write failure, preserving all attempts and adding no model rounds. All patches are source-policy compliant. Replay preserves all 64 complete-acceptance decisions; four rejected dtype attempts change partial observations. Responses report US$2.5637; conservative accounting is US$2.5728.

An original CSV attempt passes 6/6 historical initial checks but fails all restarts and current-format cases after resaving. In a separate 12-attempt development comparison, genuine packaging/attrs upstream tests and our evaluator agree: two accepts, ten rejects. A preservation grid times out for one patch already rejected by a completed regression test; timeout is not counted as semantic failure.

5. Related work

Compatibility testing predates coding agents. DUPTester studies distributed-system upgrades [7]. OpenHands' persistence CI uses released-version payloads, golden fixtures and release cutoffs [8]. SWE-bench includes historical ECSV compatibility [9], while GitChameleon studies version-conditioned API generation [10]. Our scope is repairing readers against immutable artifacts and declared continued-use behavior.

Repair overfitting [11] motivates evaluation beyond construction tests. SWE-Gate separates functional success from further constraints [12]; UTBoost strengthens SWE-bench tests [13]. Mokav and ChangeGuard compare program behaviors [14,15], and metamorphic testing studies execution relations [16]. Our save/reload and differential checks use established ideas.

SWE-Edit studies editing interfaces and downstream repair outcomes [17]. We introduce no editing algorithm: the controlled comparison tests this resource's diagnostic use. Likewise, both motivating serialization defects and their upstream fixes predate our calibrations. The contribution is the executable acceptance boundary and retained evidence.

6. Availability and limits

Resource card. Eight repair lineages, six Python projects, 65 files and 252 dependent observations. The original 32 attempts, new 64-attempt comparison and separate 54-scenario signature calibration retain their own denominators. Producer/reader source, patches, expectations, failures, reviews and licenses are included. Original contributions use Apache-2.0; third-party terms are retained. Frozen download: https://github.com/GuilinDev/historical-data-repair-contracts/releases/tag/v0.1.2

Start with python3 compat.py integrity, then python3 reproduce_ablation.py --records-only. These Python 3.10+ checks need neither Docker nor ARM64; they verify identities and recorded decisions, not fresh repair behavior. Re-execution is qualified on Linux ARM64 with Docker; after bootstrap, python3 reproduce_ablation.py replays every new patch without an API. Separate Node 22+ and independent Python checks reproduce all 725 signature-calibration observations.

Original controls reproduce 756 observations on a fresh installation; the original 32 patches replay 1,008 observations. The new campaign replays 2,016. Frozen expectations and source admissibility precede model calls. Agents see only public examples and designated source; evaluation uses network-disabled, read-only containers with one CPU and 1 GB memory. Public logs declare author-path redactions with original/exported hashes; source patches remain exact.

Limits. Selected public lineages do not estimate defect prevalence or general coding ability. The source allowlist supplies localization and excludes dependency/native-code repairs. Two joblib tasks share a project. Only the numeric case demonstrates multiple valid repair policies; the signature case tests a different rejection boundary. The numeric calibration excludes arbitrary extension dtypes. Finite checks cover declared behavior, not all APIs or security properties. Other repair-execution architectures remain unqualified.

7. Conclusion

• Continued use and persistence belong in historical-data acceptance contracts.

• Calibration admits value-preserving alternatives and rejects unsafe ambiguity; neither literal equality nor unconditional compatibility is sufficient.

• The new controlled Gemma comparison does not improve repairs with the tested span interface. Complete results, including failures, make that diagnosis reproducible.

Disclosure

Codex assisted with research, implementation, experiments, auditing and writing; Gemma generated the evaluated patches. The author is responsible for the claims and artifacts.

References

[1] Earl T. Barr et al. (2015). The Oracle Problem in Software Testing: A Survey. IEEE TSE 41(5):507-525. https://doi.org/10.1109/TSE.2014.2372785

[2] Theelx (2026). Fix bug with pandas dtypes decoding improperly. jsonpickle PR 634. https://github.com/jsonpickle/jsonpickle/pull/634

[3] William M. McKeeman (1998). Differential Testing for Software. Digital Technical Journal 10(1):100-107. https://www.cs.tufts.edu/comp/150FP/archive/bill-mckeeman/DifferentailTesting.pdf

[4] Vercel AI SDK contributors (2026). fix(ai): use injective serialization for tool approval HMAC payload. PR 17493. https://github.com/vercel/ai/pull/17493

[5] Gemma Team (2026). Gemma 4 Technical Report. arXiv:2607.02770. https://arxiv.org/abs/2607.02770

[6] Elan Markowitz et al. (2026). Google - The Gemma 4 Developer Agent Competition. Kaggle. https://www.kaggle.com/competitions/gemma-4-developer-agent

[7] Yongle Zhang et al. (2021). Understanding and Detecting Software Upgrade Failures in Distributed Systems. SOSP 2021. https://doi.org/10.1145/3477132.3483577

[8] OpenHands software-agent-sdk contributors (2026). test(ci): detect persisted settings compatibility regressions. OpenHands PR3298. https://github.com/OpenHands/software-agent-sdk/pull/3298

[9] Carlos E. Jimenez et al. (2024). SWE-bench: Can Language Models Resolve Real-World GitHub Issues? ICLR 2024. https://arxiv.org/abs/2310.06770

[10] Diganta Misra et al. (2026). GitChameleon 2.0: Evaluating AI Code Generation Against Python Library Version Incompatibilities. ACL 2026. https://aclanthology.org/2026.acl-long.2170/

[11] Edward K. Smith et al. (2015). Is the Cure Worse Than the Disease? Overfitting in Automated Program Repair. ESEC/FSE. https://doi.org/10.1145/2786805.2786825

[12] Xin He et al. (2026). SWE-Gate: Passing Functional Tests Is Not Enough for Software Engineering Agents. arXiv:2609.04167. https://arxiv.org/abs/2609.04167

[13] Boxi Yu et al. (2025). UTBoost: Rigorous Evaluation of Coding Agents on SWE-Bench. ACL 2025. https://aclanthology.org/2025.acl-long.189/

[14] Khashayar Etemadi et al. (2025). Mokav: Execution-driven differential testing with LLMs. JSS 230:112571. https://doi.org/10.1016/j.jss.2025.112571

[15] Lars Gröninger et al. (2025). ChangeGuard: Validating Code Changes via Pairwise Learning-Guided Execution. PACMSE/FSE 2025. https://doi.org/10.1145/3715760

[16] Sergio Segura et al. (2016). A Survey on Metamorphic Testing. IEEE TSE 42(9):805-824. https://doi.org/10.1109/TSE.2016.2532875

[17] Yikai Zhang et al. (2026). SWE-Edit: Rethinking Code Editing for Efficient SWE-Agent. arXiv:2604.26102. https://arxiv.org/abs/2604.26102

Software acknowledgment

Figure preparation used Scientific Agent Skills: A Library of Procedural Knowledge for Research Agents (Kassis et al., 2026). https://doi.org/10.48550/arXiv.2609.00065